

Adaptation Project: Make Your Computer-Use System Demoable Against a Live Target

Format: Adapt your existing system + a working end-to-end demo + a short write-up **Timeline:** This brief goes out **Wednesday, Aug 26**. You build **remotely** between now and Friday, and come in once — **demo day, Friday, Aug 28**, in person at 44 Montgomery Street, San Francisco — where you'll demo this project live to our leadership, alongside the take-home system it builds on. (For context: our session earlier this month ran as a two-day onsite — a full build day against a target handed out that morning, then a demo day. Yours is the one-day format: you get the target in advance and build wherever you like.) Arrive with everything working. **Builds on:** the computer-use system you built for the take-home — the *discover* → *record a reusable capability* → *deterministic replay* engine, with its guardrails, evidence, and escalation. We'll call that **your core**. **What we're evaluating:** How cleanly your core *adapts* to the concrete legacy target we provide, how completely it covers that target's surface, and how well you make the whole thing **demoable** — capabilities exposed as an invocable API, driven by a chatbot, with runs you can watch on a dashboard.

1. Context

In the take-home you built the load-bearing pieces: an LLM discovers how to accomplish a task on a UI-only app, you capture that as a **typed, versioned capability artifact**, and you **replay it deterministically** (no model in the decision loop) with typed inputs and outputs — plus safety guardrails, evidence, and a pause-and-escalate path.

This project is about proving that core is *real* by pointing it at a specific, messy legacy target, covering its **full function surface**, and wrapping it so anyone can drive it and watch it work:

capabilities recorded once → exposed as an API an agent can call → invoked from a chatbot → results and evidence visible on a dashboard.

Adapting to a new concrete surface should be a **configuration + adapter** exercise, not a rewrite. If it turns into a rewrite, that's a useful signal about where your core is too coupled to your original target — call it out in the write-up.

This brief is intentionally light on prescription. Where it doesn't dictate an answer, **make a decision and tell us why.**

2. The target application

Everything runs against the sample app we host:

➡ web-sample.interface-hiring.com

MERIDIAN CORE is a deliberately period-accurate stand-in for a US credit-union member-servicing console: server-rendered HTML, table-based layout, a numbered menu, **no test IDs**, **no clean component boundaries**, and a **per-transaction hidden token** that must be read off the page before you submit. It's exactly the "legacy, no clean DOM" surface the take-home describes — no API, the UI is the only way in.

Demo operators (no real credentials or PII — hammer on it freely):

Operator	Password	Role
teller1	password	teller
super1	password	supervisor (can perform restricted actions)

Seed members: 100234, 100987, 101555, 102777, 103001 (one member has a share already on **HOLD**).

2.1 Functions to cover (record a capability for each)

- **Sign on / session** — operator ID, password, branch; sessions time out on idle.
- **Member inquiry / selection** — search by member number *or* by last name.
- **Member record / balance** — read shares, balances, and status.
- **Funds Transfer** — from-share, to-share, amount, memo → **review** → **post** (irreversible).
- **Open New Share** — share type, initial deposit → **review** → **post**.
- **Update Member Information** — email, phone, mailing address.
- **Place Account Hold** — share, reason code, notes → **review** → **post**; **risky/irreversible and requires supervisor override**.

At minimum you must cover **check a member's balance** and **transfer money**; go through the rest of the surface too.

2.2 Runtime & exceptional states your replay must handle

These are the load-bearing part — the same error taxonomy your core already reasons about, now hitting a real target. You can force any of them per-request by appending `?inject=<kind>` to a URL (e.g. `/members/100234/transfer?inject=maintenance`), globally via the **System Settings** screen, or randomly via a configurable error rate.

inject kind	HTTP	Models
validation	400	Field/transaction rejection
notfound	404	Member record not found
permission	403	Supervisor override required
timeout	440	Session expired mid-flow
maintenance	503	Unexpected maintenance interstitial
server	500	Hard application error

Natural (non-injected) errors also occur: bad login, overdraw on a transfer, invalid email/phone on update, a hold attempted by a non-supervisor, and idle session timeout. Your replay must **distinguish expected business outcomes** ("no such member," "insufficient funds") **from recoverable conditions** (dismiss a known interstitial, retry a transient fault) **from hard failures**, and report each deliberately — exactly the contract your core already defines.

3. What to build (must-have)

3.1 Record a capability for every function

Use your discovery loop to produce a **capability artifact** for each function in §2.1 against MERIDIAN CORE, then rely on **deterministic replay** as the production path. This is the direct test of whether your core adapts: reading the per-transaction token, walking the review→post confirmation steps, and handling the supervisor-gated action should all fall out of your existing schema and replay engine.

3.2 Expose your capabilities as an API

Turn your recorded capabilities into a **callable catalog** — a programmatic surface (endpoints or a tool/function-calling interface) where an agent invokes a capability *by name with typed args* and gets a **structured result back**, without knowing anything about the underlying UI. Under the hood each invocation runs the deterministic replay. This is the take-home's *agent-facing capability interface* stretch goal, now a requirement.

3.3 A simple chatbot that drives the API

A minimal conversational front door — standing in for the AI agent — that turns a user request into the right capability invocation(s), calls your API, and **clearly confirms success or reports the error/escalation** in plain language, surfacing the structured result (confirmation numbers, balances, why it stopped). Keep it thin: it's a demo driver over your API, not a second product.

3.4 A simple dashboard to display processing

A lightweight UI to **watch the system work**: the capability catalog, run history (discovery *and* replay), each run's inputs and structured outputs, its status (success / business outcome / recoverable / failed / **escalated**), and the **evidence** your core already emits (steps, screenshots, DOM snapshots, timings, logs). This is how a reviewer sees what happened and debugs it.

3.5 Preserve your core's guarantees

Your **safety guardrails** (allowlist of permitted routes/actions, conservative handling of risky/irreversible actions like Place Hold and supervisor override, no persisting secrets or raw PII), your **evidence**, and your **pause-and-escalate / handoff** path must remain intact through the new API/chatbot/dashboard surface. Don't let the wrapper become a way around the guardrails.

3.6 Demoable end to end

The headline outcome — this is what you'll show live on Friday: someone opens the chatbot or dashboard, asks for a task, watches your system drive MERIDIAN CORE via a replayed capability, and sees a correct structured result — **including at least one run that hits an exceptional state** (bad input, not-found, an injected fault, or a teller attempting a supervisor-only Place Hold) that is detected and reported cleanly, and ideally one that **escalates**.

4. Explicitly your call

Reuse whatever you chose for your core and defend it. We are **not** prescribing:

- **Language, runtime, frameworks** — reuse your core's stack.
- **Computer-use technology** — Playwright/Puppeteer/Selenium, accessibility tree, screenshots + coordinates, a CUA/agent SDK, etc. Whatever your core already uses.
- **How much the LLM is in the loop** — discovery-time only vs. any assisted fallback.
- **The capability API / task contract** — how a capability is named, invoked, and what it returns.
- **Chatbot and dashboard tech** — keep both intentionally simple.
- **Architecture** — single process vs. services, sync vs. queued. Simpler is fine if justified.

If you can't get live LLM or browser access working for part of the flow, **mock that boundary cleanly** and document it — a well-designed seam beats a stalled project. And if you need an API key or model access for the build, tell us — we'll arrange it.

5. Scope & expectations

This is a focused adaptation sprint on top of what you already have — not a rebuild — and we assume AI-assisted development. The scaffolding (a capability API, a thin chatbot, a basic dashboard) comes together fast, so the real test is **integration and judgment**: how robustly your recorded capabilities replay against this specific messy legacy UI, how you handle the runtime errors it throws, and how coherently the pieces fit end to end.

Depth over breadth:

- **Go deep where it matters** — reliable replay against MERIDIAN CORE (the per-transaction token, review→post steps, supervisor gating) and its exceptional states.
 - **Cut depth, not whole capabilities.** Prefer a thin-but-real version of every must-have over a polished subset. It's fine to keep the chatbot or dashboard minimal, or to stub something at a clean seam — as long as it's intentional, documented, and the seam is real.
 - **Say what you cut and why**, and what you'd do next with more time.
-

6. Deliverables

Bring all of this ready to show on demo day (Friday, Aug 28):

1. **Source code** (your existing repo or a clearly linked branch), with a README covering:
 - how to set up and run the capability API, the chatbot, and the dashboard (include any keys/config, and how to run offline/mockbed if applicable),

- the **demo path**: exact command(s) to record/replay a capability against `web-sample.interface-hiring.com` and invoke it via the chatbot and/or dashboard to see the result.
2. **A short write-up** (Markdown, ~1–2 pages) covering:
 - what adapting to this target actually took, and anything in your core you had to change (and why),
 - how you exposed capabilities as an API and the shape of that contract,
 - how you drive this legacy UI reliably, and how you detect and handle its runtime/exceptional states,
 - how your safety, evidence, and escalation guarantees survive the new path,
 - what you deliberately left out and would build next.
 3. **A live demonstration on demo day** of the end-to-end flow — a **successful** capability run **and at least one run that hits an error/exceptional state** (and, ideally, one that escalates), reported cleanly. Bring evidence/logs (and ideally a short screen recording) as a backup in case of live-demo trouble — the network will not be your friend under pressure.
-

7. Evaluation criteria

Weighted roughly in this order:

Area	What we look for
Adaptation quality	How cleanly your core pointed at this target — a configuration/adaptor change, not a rewrite. Where it wasn't, whether you understood why.
Correctness of the core loop	Capabilities actually complete real tasks against MERIDIAN CORE and replay deterministically with correct, structured results across the full function set.
Robustness & error handling	How replay detects and responds to validation errors, not-found, permission denials, dialogs, timeouts, and transient faults; clean separation of business outcomes from recoverable conditions and hard failures.

Area	What we look for
Capability API / task contract	Clear typed inputs/outputs, sensible boundaries, a catalog an agent could invoke by name without knowing the UI.
Demoability	The chatbot + dashboard make the system easy to drive and easy to watch, including on the unhappy path.
Safety & data handling	Allowlist enforcement, conservative treatment of risky/irreversible actions (Place Hold, supervisor override), redaction of regulated financial data.
Escalation	A real "stuck → stop → escalate with context" path, preserved through the new surface.
Communication	The write-up makes your reasoning, trade-offs, and cut lines clear.

We do **not** reward feature breadth, framework name-dropping, or scaling infrastructure. A small, correct, well-argued, demoable system is the goal.

8. Ground rules

- **AI-assisted development is assumed and encouraged.** You own everything you submit and must be able to explain and defend any part of it.
- The sample app is yours to hammer on — public demo operators, no real credentials or PII. It's stateful in memory and resets on redeploy, so don't rely on data persisting.
- Keep secrets out of the repo.
- **During demo.** Whatever isn't done, document as next steps — we're evaluating judgment, not endurance. Arrive rested; we'd rather see you sharp than heroic.